

DESARROLLO WEB EN ENTORNO CLIENTE

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y
FORMULARIOS

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Evento

- Mecanismo que se acciona cuando el usuario realiza un cambio sobre la página web.
- Capturar un evento es programar una acción para que se realice una tarea.
- El encargado de gestionar los eventos es el DOM (Document Object Model).

Manejador

- Palabra reservada que indica la acción que se va a manejar. Por ejemplo, en el evento click el manejador es onClick.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos de ratón

Ratón

Evento	Descripción
Click	Cuando pulsamos el botón derecho del ratón.
Dblclick	Cuando hacemos doble click el botón derecho del ratón.
Mousedown	Cuando pulsamos un botón del ratón.
Mouseup	Cuando soltamos un botón del ratón que previamente teníamos pulsado.
Mouseout	Cuando el puntero del ratón está dentro de un elemento y este puntero es desplazado fuera del elemento.
Mouseover	Cuando el puntero del ratón se encuentra fuera de un elemento, y este se desplaza hacia el interior.
Mousemove	Cuando el puntero del ratón se encuentra dentro de un elemento. Es importante señalar que este evento se producirá continuamente una vez tras otra mientras el puntero del ratón permanezca dentro del elemento.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos de teclado

Teclado

Evento	Descripción
Keydown	Cuando pulsamos una tecla del teclado. Si mantenemos pulsada una tecla de forma continua, el evento se produce una y otra vez hasta que soltemos la misma
Keypress	Cuando pulsamos una tecla de un carácter alfanumérico (el evento no se produce si pulsamos enter, la barra espaciadora, etc...). En el caso de mantener una tecla pulsada, el evento se produce de forma continuada.
Keyup	Cuando soltamos una tecla.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos de toque

Teclado

Evento	Descripción
Touchmove	Cuando pulsamos una tecla del teclado. Si mantenemos pulsada una tecla de forma continua, el evento se produce una y otra vez hasta que soltemos la misma
Touchmove	Cuando pulsamos una tecla de un carácter alfanumérico (el evento no se produce si pulsamos enter, la barra espaciadora, etc...). En el caso de mantener una tecla pulsada, el evento se produce de forma continuada.
Keyup	Cuando soltamos una tecla.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos HTML

HTML

Evento	Descripción
Load	El evento load hace referencia a la carga de distintas partes de la página. Éste se produce en el objeto Window cuando la página se ha cargado por completo. En el elemento actúa cuando la imagen se ha cargado. En el elemento <object> se acciona al cargar el objeto completo.
Unload	El evento unload actúa sobre el objeto Window cuando la página ha desaparecido por completo (por ejemplo, si pulsamos el aspa cerrando la ventana del navegador). También se acciona en el elemento <object> cuando desaparece el objeto.
Abort	Este evento se produce cuando el usuario detiene la descarga de un elemento antes de que haya terminado, actúa sobre un elemento <object>.
Error	El evento error se produce en el objeto Window cuando se ha producido un error en JavaScript. En el elemento cuando la imagen no se ha podido cargar por completo y en el elemento <object> en el caso de que un elemento no se haya cargado correctamente.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos HTML

HTML

Evento	Descripción
Change	Cuando los cuadros de texto <code><input></code> y <code><textarea></code> pierden el foco y el contenido que tenían ha variado. Cuando un elemento <code><select></code> cambia de valor.
Submit	Cuando pulsamos sobre un botón de tipo submit en un formulario
Reset	Cuando pulsamos sobre un botón de tipo reset en un formulario.
Error	El evento error se produce en el objeto Window cuando se ha producido un error en JavaScript. En el elemento <code></code> cuando la imagen no se ha podido cargar por completo y en el elemento <code><object></code> en el caso de que un elemento no se haya cargado correctamente.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.1. MODELO DE GESTIÓN DE EVENTOS

Eventos HTML

HTML

Evento	Descripción	
Resize	Cuando redimensionamos el navegador, actúa sobre el objeto Window.	
Scroll	Cuando varía la posición de la barra de scroll en cualquier elemento que la tenga	
Focus	Cuando un elemento obtiene el foco. En elementos button, input, label, select, textarea, body.	
Blur	Cuando un elemento pierde el foco. En elementos button, input, label, select, textarea, body.	

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Eventos DOM

DOM

Evento	Descripción
DOMSubtreeModified	cuando añadimos o eliminamos nodos en el subárbol de un elemento o documento.
DOMNodeInserted	cuando añadimos un nodo hijo a un nodo padre.
DOMNodeRemoved	cuando eliminamos un nodo que tiene nodo padre.
DOMNodeRemovedFromDocument	cuando eliminamos un nodo del documento.
DOMNodeInsertedIntoDocument	cuando añadimos un nodo al documento.
DOMContentLoaded	cuando se carga el árbol DOM. (Algo antes que evento "load")

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario:

- `<html>`
 - `<head>`
 - `<title>Mi formulario</title>`
 - `</head>`
 - `<body>`
 - `<h3>Formulario</h3>`
 - `<form action= "pagina.php" method="post">`
 - `//Elementos`
 - `</form>`
 - `</body>`
 - `</html>`
-
- Action: es la url de la página donde se envían los datos que el formulario ha recopilado cuando se validan.
 - Method: puede ser POST o GET, dependiendo del enmascaramiento de los datos enviados.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

Propiedad	Descripción
acceptCharset	Ajusta o devuelve el valor del atributo accept-charset en un formulario.
action	Ajusta o devuelve el valor del atributo Action en un formulario.
enctype	Ajusta o devuelve el valor del atributo enctype en un formulario.
length	Devuelve el número de elementos de un formulario.
method	Ajusta o devuelve el valor del atributo method en un formulario.
name	Ajusta o devuelve el valor del atributo name en un formulario.
target	Ajusta o devuelve el valor del atributo target en un formulario.

<form>

Método	Descripción
reset()	Resetea un formulario.
submit()	Envía un formulario.

form.elements[]

- Devuelve un array con todos los input de un formulario.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

<input>

Propiedad	Descripción
Type	Puede ser, a su vez: • Text (cuadro de texto). • Password (cuadro de contraseña). • Checkbox (casilla de verificación). • radio (opción de entre dos o más). • Submit (botón de envío del formulario). • Reset (botón de vaciado de campos). • File (botón para buscar ficheros). • Hidden (campo oculto para, el usuario no lo visualiza en el formulario) • Image (botón de imagen en el formulario). • Button (botón del formulario).
Name	Asigna un nombre al elemento. Es necesario para que el servidor pueda trabajar con él.
Value	Inicializa el valor del elemento.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

<input>

Propiedad	Descripción
Size	Tamaño inicial. En campos text y password se refiere al número de caracteres.
Maxlength	Número máximo de caracteres que pueden contener text y password.
Checked	Para elementos checkbox y radio. Indica si el checkbox está marcado; o cuál de los botones de radio está pulsado.
Disabled	El elemento aparece deshabilitado. El dato no se envía al servidor.
Readonly	Bloquea el contenido de control: el valor del elemento no se puede modificar (solo lectura).
Src	Solo se utiliza para asignar la url de una imagen que se colocará en como botón.
Alt	Descripción del elemento, que se visualiza si nos ponemos con el ratón sobre él.
Required	Indica si el campo debe ser completado antes de enviar el formulario.

Método	Descripción
select()	Selecciona el contenido de un campo de texto.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Cuadro de texto. Ejemplo:

- `< input type="text" name="nombre" />`

Cuadro de contraseña. Ejemplo:

- `< input type="password" name="contrasena" />`

Campo oculto. Ejemplo:

- `< input type="hidden" name="campoOculto" value="valor"/>`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Casilla de verificación. Ejemplo:

- `<p>Frutas</p>`
- `<input name="fresa" type="checkbox" value="fresa"/>Fresa<br/>`
- `<input name="platano" type="checkbox" value="platano"/>Plátano <br/>`
- `<input name="manzana" type="checkbox" value="manzana"/>Manzana <br/>`

Radio button. Ejemplo:

- `<p>Frutas</p>`
- `<input name="fruta" type="radio" value="fresa"/>Fresa<br/>`
- `<input name="fruta" type="radio" value="platano"/>Plátano <br/>`
- `<input name="fruta" type="radio" value="manzana"/>Manzana <br/>`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Botón de envío. Ejemplo:

- `<input name="enviar" type="submit" value="Enviar formulario"/>`

Botón de reset. Ejemplo:

- `<input name="enviar" type="reset" value="Borrar formulario"/>`

Botón de imagen. Ejemplo:

- `<input name="enviar" type="image" src="imagen.jpg"/>`

Botón. Ejemplo:

- `<input name="enviar" type="button" value="Botón de prueba"/>`

Selector de ficheros. Ejemplo:

- Fichero adjunto: `<input type="file" name="fichero" />`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Existen también los elementos `button` que funcionan de manera similar, aunque al tratarse de elementos con etiquetas de apertura y cierre podemos introducir cualquier tipo de código HTML.

Botón de envío. Ejemplo:

- `<button type="submit">Enviar formulario</button>`

Botón de reset. Ejemplo:

- `<button type="reset">Borrar formulario</button>`

Botón neutro. Ejemplo:

- `<button type="button">Botón 1</button>`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

<label>

Propiedad	Descripción
for	Indica a qué otro elemento del formulario está asociada.

Label. Ejemplo:

```
<label for =“nombre”>Nombre</label>  
<input type=“text” name=“nombre” />
```

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

`<textarea>`

Propiedad	Descripción
Name	Asigna un nombre al elemento. Es necesario para que el servidor pueda trabajar con él.
Rows	Numero de líneas del textarea.
Cols	Ancho del textarea.
Maxlength	Número máximo de caracteres que pueden contener text y password.
Checked	Para elementos checkbox y radio. Indica si el checkbox está marcado; o cuál de los botones de radio está pulsado.
Disabled	El elemento aparece deshabilitado. El dato no se envía al servidor.
Readonly	Bloquea el contenido de control: el valor del elemento no se puede modificar (solo lectura).
Required	Indica si el campo debe ser completado antes de enviar el formulario.

Textarea. Ejemplo:

```
<textarea name="comentarios" rows="4" cols="30" ></textarea>
```

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

`<select>` y `<option>`

Propiedad	Descripción
Name	Asigna un nombre al elemento. Es necesario para que el servidor pueda trabajar con él.
Disabled	El elemento aparece deshabilitado. El dato no se envía al servidor.
Multiple	Permite marcar más de una fila visible. Requiere del atributo size.
Size	En caso de que múltiple esté marcado, muestra cuántas filas estarán visibles.
Required	Indica si el campo debe ser completado antes de enviar el formulario.

Dentro del elemento `<select>` se encuentra la etiqueta `<option>` con el campo `value` para almacenar el valor y en su interior lo que se visualizará.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Elementos de un formulario

`<fieldset>` y `<legend>`

Propiedad	Descripción
Name	(Fieldset) Asigna un nombre al elemento.
Align	(Legend) Permite elegir la alineación del texto del legend sobre el fieldset.

Fieldset se utiliza para agrupar elementos del formulario mediante una línea y permite asignar un título para esta agrupación gracias a la etiqueta `<legend>`

Fieldset. Ejemplo:

```
<form>
  <fieldset>
    <legend>Datos:</legend>
    Nombre: <input type="text" size="30"><br>
    Email: <input type="text" size="30"><br>
  </fieldset>
</form>
```

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Desplegable. Ejemplo:

- `<p>Comida: </p>`
 - `<select name="comida">`
 - `<option value="pasta">Pasta</option>`
 - `<option value="carne">Carne</option>`
 - `<option value="pescado">Pescado</option>`
 - `<option value="postre">Postre</option>`
 - `</select>`

Desplegable con elección múltiple. Ejemplo:

- `<p>Comida: </p>`
 - `<select multiple size="3" name="comida[]">`
 - `<option value="pasta">Pasta</option>`
 - `<option value="carne">Carne</option>`
 - `<option value="pescado">Pescado</option>`
 - `<option value="postre">Postre</option>`
 - `</select>`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

Ejemplo de formulario:

```
<html>
<head>
  <title>Formulario</title>
</head>
<body>
  <h1>Formulario</h1>
  <form action="procesar.php" method="post" >
    <table>
      <tr><td>Nombre: </td><td><input type="text" name="nombre"/></td></tr>
      <tr><td>Password: </td><td><input type="password" name="psw" /></td></tr>
      <tr>
        <td>Sexo : </td>
        <td>
          <input type="radio" name="sexo" value="H" checked /> Hombre
          <input type="radio" name="sexo" value="M" /> Mujer
        </td>
      </tr>
      <tr><td>Mayor de 18:</td><td><input type="checkbox" name="mayor" checked/></td></tr>
      <tr><td>Fichero: </td><td><input type="file" name="upfile" /></td></tr>
      <tr><td>Hidden: </td><td><input type="hidden" name="escondido" value="sorpresa" /></td></tr>
    </table>
  </form>
</body>
</html>
```

The screenshot shows a web form titled "Formulario" with the following fields and controls:

- Nombre:** A text input field.
- Password:** A password input field.
- Sexo :** Two radio buttons labeled "Hombre" (selected) and "Mujer".
- Mayor de 18:** A checked checkbox.
- Fichero:** A file upload control showing "Examinar..." and "No se ha seleccionado ningún archivo."
- Hidden:** A hidden input field.
- Comida:** A dropdown menu with "Pasta" selected.
- Bebidas:** A list box with "Agua", "Vino", and "Cerveza".
- Buttons:** "Enviar" and "Borrar" buttons at the bottom.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.2. UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO

```
<tr><td>Comida: </td>
<td>
<select name="comida">
<option value="pasta">Pasta</option>
<option value="carne">Carne</option>
<option value="pescado">Pescado</option>
<option value="postre">Postre</option>
</select>
</td></tr>
<tr><td>Bebidas: </td>
<td>
<select multiple size="3" name="bebida[]">
<option value="agua">Agua</option>
<option value="vino">Vino</option>
<option value="cerveza">Cerveza</option>
</select>
</td></tr>
</table>
<p>
<input type="submit" value="Enviar" />
<input type="reset" value="Borrar" />
</p>
</form>
</body>
</html>
```



The screenshot shows a web form titled "Formulario". It contains the following fields and controls:

- Nombre:** A text input field.
- Password:** A password input field.
- Sexo :** Radio buttons for "Hombre" (selected) and "Mujer".
- Mayor de 18:** A checkbox that is checked.
- Fichero:** A file upload control with a button labeled "Examinar..." and a message "No se ha seleccionado ningún archivo."
- Hidden:** A hidden input field.
- Comida:** A dropdown menu with "Pasta" selected.
- Bebidas:** A list box showing "Agua", "Vino", and "Cerveza".
- Buttons:** "Enviar" (submit) and "Borrar" (reset).

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Seleccionar un formulario

- Disponemos del siguiente formulario:
 - `<form id="miFormulario" name="miFormulario" action="siguiente.php"...>`

`getElementById()`

- `var formulario = document.getElementById("contactar");`

`getElementsByTagName()`

- `var formularios = document.getElementsByTagName("form");`
- `var primerform = formularios[0];`

O en una sola línea:

- `var primerform = document.getElementsByTagName("form")[0];`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Seleccionar un formulario

- Disponemos del siguiente formulario:
 - `<form id="miFormulario" name="miFormulario" action="siguiente.php"...>`

`forms[0]`

- `var formularios = document.forms; //Devuelve un array de formularios`
- `var primerform = formularios [0];`

O en una sola línea:

- `Var formularios = document.forms[0];`

`forms["nombre"]`

- `var formularios = document.forms; //Devuelve un array de formularios`
- `var primerform = formularios ["contactar"];`

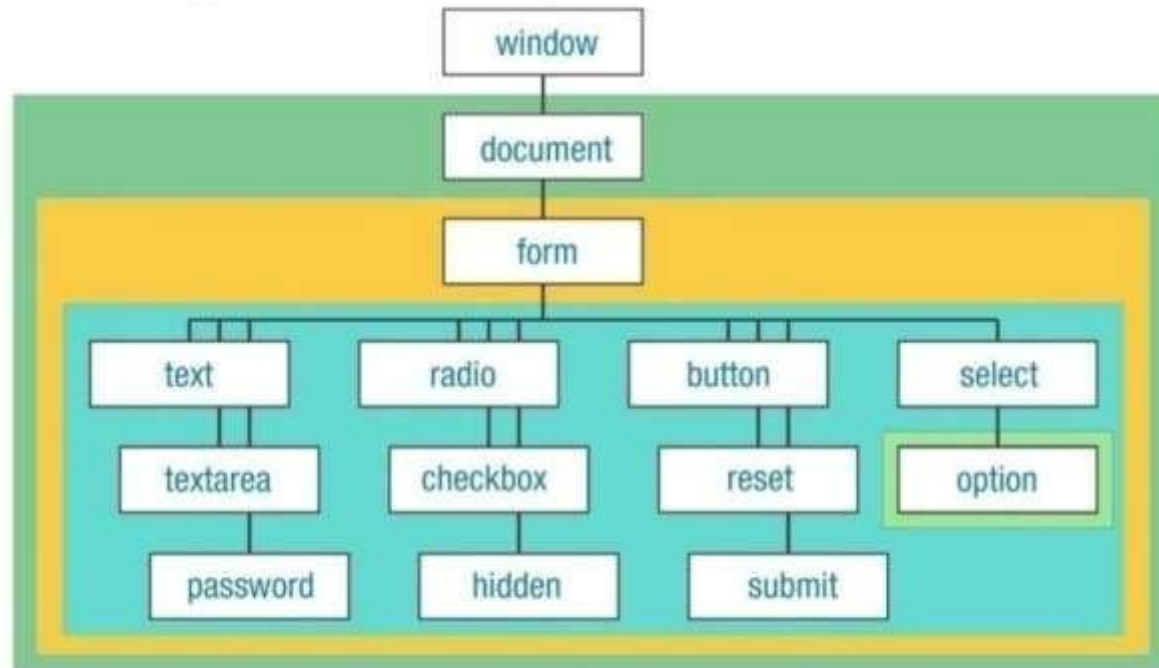
O en una sola línea:

- `Var formularios = document.forms["contactar"];`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

El formulario como contenedor de objetos:



5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Selección de objetos de un formulario

- Recuerda que...

`form.elements[]` devuelve un array con todos los input de un formulario.

`getElementById()` devuelve un elemento que tenga un id determinado.

`getElementsByTagName()` devuelve un array de elementos de un tipo.

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Selección de objetos de un formulario

- En este ejemplo:
- `<form id="formularioBusqueda" action="siguiente.php">`
 - `<p>`
 - `<input type="text" id="entrada" name="cEntrada">`
 - `<input type="submit" id="enviar" name="enviar" value="Buscar...">`
 - `</p>`
- `</form>`
- **Para hacer referencia al input de tipo texto podemos hacerlo mediante:**
 - `document.getElementById("entrada")`
 - `document.formularioBusqueda.cEntrada`
 - `document.formularioBusqueda.elements[0]`
 - `document.forms["formularioBusqueda"].elements["cEntrada"]`
 - `document.forms["formularioBusqueda"].cEntrada`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Objetos input de tipo texto

- Es recomendable utilizar un id igual que el name para poder referirnos a ellos con getElementById.
- Podemos referirnos a cualquiera de sus propiedades para cambiarlas.
- Ejemplo:
 - `<input type="text" name="nombre" id="nombre"/>`
 - `document.getElementById("nombre").value="Valor nuevo"; //Cambia el valor del texto`

Objetos input de tipo checkbox

- La propiedad value es el texto asociado al objeto.
- El formulario solamente envía los datos si el checkbox está marcado.
- Ejemplo:
 - `<input type="checkbox" id="cantidad" name="cantidad" value="100"/>`
 - `document.getElementById("cantidad").checked=true; //Marca el checked`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Objetos input de tipo radio

- Cuando utilizamos varios radiobutton con el mismo nombre, se crea un array de objetos input radio.
- Podemos consultar el número de radio button con `formulario.gruporadio.length`;
- Podemos consultar si un radio button está marcado con `formulario.gruporadio[num].checked` y devolverá true o false.
- Ejemplo:
 - ```
for(var i=0; i<document.formulario.frutas.length;i++){
```
  - ```
    if(document.formulario.frutas[i].checked) //Comprueba que es true
```
 - ```
 alert(document.formulario.actores[i].value);
```
  - ```
}
```

Objetos select

- La propiedad `selectedIndex` devuelve el índice de la opción que ha sido seleccionada.
- Cada opción tiene dos propiedades accesibles: `text` y `value` que indican el valor visible y el valor interno, respectivamente.
- Ejemplo:
 - `<option value="ZA">Zamora</option>`
 - `formulario.nombreCampoSelect.options[n].text; //Devuelve Zamora`
 - `formulario.nombreCampoSelect.options[n].value; //Devuelve ZA`

5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

Usando this

- Utilizamos this para hacer referencia al objeto que llama a un evento.
- Ejemplo:
 - `<input type="text" name="nombre" id="nombre" value="Laura" onclick="identificar(this)"/>`
 - ```
function identificar(objeto){
 • var nombre=objeto.name;
 • var valor=objeto.value;
 • var id=objeto.id;
 • var tipo=objeto.type;
 • alert("Datos del objeto: "+nombre+ " – "+valor+ " – "+id+ " – "+tipo);
 • }
}
```



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

### Eventos

- Los eventos se ejecutan cuando el usuario interactúa con la página (haciendo clic, pulsando una tecla, moviendo el ratón, etc.) o bien se tratan de eventos no relacionados con acciones del usuario (como onload, que sucede cuando la página se carga).
- Un poco de historia:

Incompatibilidades de navegadores: había que comprobar si los métodos eran soportados.

Modelo de registros de eventos en línea: se añade como atributo a la etiqueta html:  
`<a href="..." onClick=accion/>`

Modelo de registro de eventos tradicional: los eventos son una propiedad de los elementos:  
`elemento.onclick=accion;`

Modelo avanzado de registro de eventos del W3C: utiliza `addEventListener()`;

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

### Modelo de registro de eventos en línea

- El evento se añade como atributo de una etiqueta html.
- Se ejecuta primero el script y luego la acción por defecto.
- Ejemplos:
  - 1) `<a href="siguiente.html" onclick="alert('Has pulsado en el enlace')">Pulsa aquí</a>`
  - 2) `<a href="siguiente.html" onclick="alertar()">Pulsa aquí</a>`
    - `function alertar(){alert("Has pulsado el enlace");}`
- **Para evitar que el navegador ejecute acción por defecto:**
  - `<a href="siguiente.html" onclick="alertar(); return false;">Pulsa aquí</a>`  
De este modo no ejecutaría la acción por defecto: redirigir a la página siguiente.html.
- **Para preguntar si ejecutar o no la acción por defecto:**
  - `<a href="siguiente.html" onclick="return preguntar()">Pulsa aquí</a>`
  - `function preguntar(){return confirm("¿Desea realmente ir a esa dirección?");}`

## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

¿Por qué no utilizar el modelo de registro de eventos en línea?

No separa la programación de la estructura.

Si la función redirige a una página no se ejecutan las demás tareas.

Al ejecutar la acción por defecto se pueden perder los objetos que hay en memoria (ej. Al cambiar de página).

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

### Modelo de registro de eventos tradicional



- **El evento se convierte en una propiedad del elemento:**
- **Ejemplos:**
  - `elemento.onclick = acción;`
  - `function accion() {...}`
- **Para eliminar un gestor de eventos de un elemento:**
  - `elemento.onclick=null;`
- **Para llamar al gestor de eventos cuando queramos:**
  - `elemento.onclick();`
- No usamos paréntesis, porque no necesitamos que devuelva el resultado.  
~~`elemento.onclick=accion();`~~ `elemento.onclick=accion;`

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.3. MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

### Modelo de registro de eventos avanzado del W3C



- **Utiliza un evento `addEventListener()` con tres argumentos:**
  - Tipo de evento.
  - Función a ejecutar.
  - Valor booleano para elegir la fase de captura (`true`) o burbujeo (`false`).
    - `elemento.addEventListener('evento', función, false|true)`
    - Ej: `document.getElementById("mienlace").addEventListener('click', alertar, false);`
  - ¡¡Ojo!! No lleva “on” delante del nombre del evento.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.3 MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

### Modelo de registro de eventos avanzado del W3C



- **Eliminar un evento de un elemento:**
  - `elemento.removeEventListener();`
- **Crear más de un evento para el mismo elemento:**
  - `document.getElementById("mienlace").addEventListener('click', alertar, false);`
  - `document.getElementById("mienlace").addEventListener('click', avisar, false);`
  - `document.getElementById("mienlace").addEventListener('click', chequear, false);`
- **Crear un evento anónimo (no está asociado a ninguna función):**
  - `element.addEventListener('click', function () {  
    this.style.backgroundColor = '#cc0000'; }, false)`

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Instancia de objeto Event

Por defecto, si así lo indicamos en la declaración de la función que se ejecuta al producirse un evento, se recibe un parámetro con una instancia del propio evento que se disparó.

Mediante este parámetro podemos obtener mucha información del evento como el elemento que lo disparó (`Event.target`), el tipo de evento que se disparó (`Event.type`) o la tecla que se pulsó en un evento de tipo teclado (`Evento.key`), entre otros varios.

```
elemento.addEventListener("manejador", (e) => {...});
```

El nombre que se asigna a este parámetro puede ser cualquiera que queramos indicar pero suelen usarse “e” o “event”.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Descarga y ejecución de archivos .js

Por defecto, los archivos .js se descargan paralizando el parseo o análisis del HTML para ello, una vez descargado se ejecuta y tras la ejecución continúa el parseo HTML.

Ésto puede provocar que se reference desde el script a partes del HTML que aún no existen y provocar, por tanto, errores.

Sin embargo existen métodos para evitar esto:

- Atributo de etiqueta script async (no siempre es una solución)
- Atributo de etiqueta script defer
- Evento DOMContentLoaded
- Evento Load



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Descarga y ejecución de archivos .js

Si se establece el atributo **async**, el script se descarga en paralelo al análisis de la página y se ejecuta tan pronto como esté disponible. El análisis de la página se interrumpe una vez que el script se descarga por completo y luego se ejecuta antes de continuar con el análisis del resto de la página.

Si se establece el atributo **defer**, se especifica que el script se descarga en paralelo al análisis de la página y se ejecuta después de que la página haya terminado de analizarse.



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Descarga y ejecución de archivos .js

Si usamos la gestión de eventos con `DOMContentLoaded` sobre el documento y `Load` sobre la ventana nos aseguraremos que el árbol DOM y la ventana están cargados.

- `document.addEventListener("DOMContentLoaded", () => {...});`

Cuando el árbol DOM está totalmente cargado. Diferencia respecto a `defer` que éste se da cuando se ha parseado todo el HTML pero si algunos elementos crean algo por ejemplo en su `connectedCallback()` con `defer` no se habrá terminado de crear.

- `window.addEventListener("Load", () => {...});`

Cuando todos los elementos de la ventana han sido cargados. Incluidas imágenes, subframes, estilos, etc.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de formularios

- **Debemos indicar, en la etiqueta del formulario, que realice la validación:**
  - `<form action="URL" method="post" name="formValidado" onsubmit="return validar()">`
  - ...
  - `</form>`
- **Podemos hacerlo con un `AddEventListener` (mejor):**
  - `document.getElementById("enviar").addEventListener('click', validar, false);`

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de formularios

- **En primer lugar definimos una función “iniciar” o una función anónima que se va a cargar únicamente cuando el documento haya sido cargado completamente.**
  - ```
document.addEventListener("DOMContentLoaded", () => {  
  document.getElementById("enviar").addEventListener('click', validar, false);  
});
```
- **La función validar recibe el evento por defecto asociado al botón enviar. Validamos cada apartado llamando a las funciones correspondientes:**
 - ```
function validar(eventopordefecto) {
 if (validarcampotexto(this) && validarMatricula() && validarProvincia() &&
 confirm("¿Deseas enviar el formulario?"))
 return true;
 else {
 eventopordefecto.preventDefault(); //Cancelamos el evento de envío por defecto asignado al botón
 return false; // Salimos de la función devolviendo false.
 }
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de un campo como obligatorio:

**Valida un campo de texto:**

```
function validacion() {
 valor =
 document.getElementById("campo").value;

 if(valor == "" || valor.length== 0){
 alert("El campo no puede ser
vacío");
 return false;
 }
 return true;
}
```

**Valida todos los campos de texto del formulario:**

```
var formulario = objeto.form;
for (var i=0; i<formulario.elements.length; i++)
{
 if (formulario.elements[i].type == "text" &&
 formulario.elements[i].value=="")
 {
 alert("El campo:
"+formulario.elements[i].name+" no puede
estar en blanco");
 formulario.elements[i].className="error";
 formulario.elements[i].focus();
 return false;
 }
}
```

## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

#### Validación de un campo como obligatorio:

Utilizando HTML podemos hacerlo añadiendo la propiedad **required** al input en cuestión.

```
<input type="text" required />
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de un campo de texto como numérico:

#### Ejemplo 1 (teléfono):

```
function validaNum() {
 valor =
 document.getElementById("telefono").value;
 if(isNaN(valor)) {
 alert("El campo tiene que ser
numérico");
 return false;
 }
 return true;
}
```

#### Ejemplo 2 (edad):

```
if (formulario.elements[i].id=="edad"){
 if (isNaN(formulario.elements[i].value) ||
 formulario.elements[i].value <0 ||
 formulario.elements[i].value >105) {
 alert("El campo: "+formulario.elements[i].name+"
posee valores incorrectos o la edad <0 o
>105");
 }
}
```

## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

#### Validación de un campo de texto como numérico:

Utilizando HTML podemos indicar que el input es de tipo numérico y podemos también limitar el valor mínimo y máximo que se puedan introducir usando las propiedades **min** y **max**.

```
<input type="number" min="1" max="5">
```



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de un campo de fecha:

#### Ejemplo:

```
function validaFecha() {

 vardia= document.getElementById("dia").value;

 varmes =
 document.getElementById("mes").value;

 varano =
 document.getElementById("ano").value;

 fecha = new Date(ano, mes, dia);

 if(!isNaN(fecha)) {
 return false;
 }
 return true;
}
```



## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

#### Validación de un campo de fecha:

También se puede establecer un mínimo y máximo para un campo de entrada de tipo fecha usando también las propiedades **min** y **max**.

```
<input type="date" max="2000-01-02 min="1979-12-31">
```

## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

#### Modificación de la apariencia de un campo erróneo:

Podemos modificar la apariencia de un campo creando en css una clase error:

```
.error{ border: solid 2px #FF0000; }
```

Así, cuando un campo sea erróneo, le aplicamos dicha clase.

Ejemplo:

```
if (formulario.elements[i].type == "text" &&
 formulario.elements[i].value=="") {
 alert("El campo: "+formulario.elements[i].name+" no puede estar en
 blanco");

 formulario.elements[i].className="error";

 formulario.elements[i].focus();
 return false;
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.4. ENVÍO Y VALIDACIÓN DE FORMULARIOS

### Validación de un checkbox:

#### Ejemplo:

```
function validaCheck() { elemento=
 document.getElementById("campoCondiciones ");
 if(!elemento.checked) {
 return false;
 }
 return true;
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Expresiones regulares:

- Se utiliza en datos que siguen un patrón concreto.
- Ejemplos de expresiones regulares:

Código postal.

Teléfono  
(sobre todo  
con prefijos...)

DNI

Tarjeta de  
extranjería.

Código de  
cuenta del  
banco.

Correo  
electrónico.

Identificadores  
de usuario.



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Caracteres especiales

Car.	Descripción	Detalle
^	Principio de entrada o línea.	Este carácter indica que las cadenas deberán comenzar por el siguiente carácter. Si este fuera una "a" minúscula como indicamos en el punto anterior la expresión regular sería, ^a.
\$	Fin de entrada o línea.	Indica que la cadena debe terminar por el elemento precedido al dólar.
*	El carácter anterior 0 o más veces.	* El carácter anterior 0 o más veces. El asterisco indica que el carácter anterior se puede repetir en la cadena 0 o más veces.
+	El carácter anterior 1 o más veces.	El símbolo más indica que el carácter anterior se puede repetir en la cadena una o más veces.
?	El carácter anterior una vez como máximo.	El símbolo interrogación indica que el carácter anterior se puede repetir en la cadena cero o una vez
.	Cualquier carácter individual.	El símbolo punto indica que puede haber cualquier carácter individual salvo el de salto de línea.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Caracteres especiales

Car.	Descripción	Detalle
x y	X ó y	La barra vertical indica que puede ser el carácter x o el y.
{n}	n veces el carácter anterior	El carácter anterior a las llaves tiene que aparecer exactamente n veces.
{n,m}	Entre n y m veces el carácter anterior	El carácter anterior a las llaves tiene que aparecer como mínimo n y como máximo m veces.
[abc]	Cualquier carácter de los corchetes.	En la cadena puede aparecer cualquier carácter que este incluido en los corchetes.
[^abc]	Un carácter que no esté en los corchetes	En la cadena pueden aparecer todos los caracteres que no estén incluidos en los corchetes.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Caracteres especiales

Car.	Descripción	Detalle
\b	Fin de palabra	Este símbolo indica que tiene que haber un fin de palabra o retorno de carro.
\B	No fin de palabra	El símbolo \B indica cualquiera que no sea un límite de palabra.
\d	Cualquier carácter dígito	Este símbolo indica que puede haber cualquier carácter numérico, de 0 a 9.
\D	Carácter que no es dígito	Este símbolo indica que puede haber cualquier carácter siempre que no sea numérico.
\f	Salto de página	Este símbolo indica que tiene que haber un salto de página
\n	Salto de línea	Este símbolo indica que tiene que haber un salto de línea.
\s	Cualquier espacio en blanco	Este símbolo indica que tiene que haber un carácter individual de espacio en blanco: espacios, tabulaciones, saltos de página o saltos de línea.
\S	Cualquier carácter que no sea blanco	Este símbolo indica que tiene que haber cualquier carácter individual que no sea un espacio en blanco.
\t	Cualquier tabulación	Este símbolo indica que tiene que haber cualquier tabulación.
\w	Cualquier alfanumérico	Este símbolo indica que puede haber cualquier carácter alfanumérico.
\W	Cualquier carácter no alfanumérico	Este símbolo indica que puede haber cualquier carácter que no sea alfanumérico.



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Validación de un DNI:

Ejemplo:

```
function validaDNI(){
 valor = document.getElementById("dni").value;
 var letras =
 ['T','R','W','A','G','M','Y','F','P','D','X','B','N','J','Z','S','Q','V','H','L','C','K','E','T'];
 if(!(/^d{8}[A-Z]$/.test(valor))){ return false; }
 if(valor.charAt(8) != letras[(valor.substring(0, 8))%23])
 { return false; }
 return true;
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Validación de un correo electrónico:

Ejemplo:

```
function validaEmail(){
 valor = document.getElementById("campo").value;
 if(!(/^w+([-.']w+)*@w+([-.]w+)*\w+([-
 .]w+)/.test(valor))) {
 return false;
 }
 return true;
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.5. EXPRESIONES REGULARES

### Validación de un teléfono:

Ejemplo:

```
function validaTelefono(){
 valor =
 document.getElementById("telefono").value;
 if(!(/^\d{9}$/.test(valor))) {
 return false;
 }
 return true;
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Cookie:

- Fichero de texto propio de cada navegador.
- Se almacena en diferentes lugares según el navegador y el sistema operativo.
- Se comenzaron a utilizar para guardar la información de los carritos de compra.
- Son específicas del dominio: un dominio no puede acceder a la cookie de otro dominio.
- Se envían automáticamente con cada solicitud HTTP a ese dominio.
- Tienen una fecha de caducidad: como el número de cookies es limitado, el propio navegador se encarga de borrarlas.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Vulnerabilidades comunes de las cookies:

- **XSS (Cross-Site Scripting):** Los atacantes pueden inyectar scripts maliciosos que roben cookies si éstas no están protegidas adecuadamente.
- **CSRF (Cross-Site Request Forgery):** El navegador envía cookies automáticamente, lo que permite a un atacante ejecutar acciones no deseadas en sitios donde el usuario está autenticado.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



- En Windows:  
“\Usuarios\<Tu\_usuario>\AppData\Roaming\Mozilla\Firefox\Profiles\<perfil>\cookies.sqlite”.
- En Unix/Linux:  
“~/.mozilla/firefox/<perfil>/cookies.sqlite”.
- En macOS: “~/Library/Application Support/Firefox/Profiles/<perfil>/cookies.sqlite”.

Este archivo no es legible directamente como un archivo de texto, pero puede ser explorado usando herramientas de bases de datos SQLite o extensiones especializadas para navegadores que permiten visualizar las cookies almacenadas.



- En Windows:  
“\Usuarios\<Tu\_usuario>\AppData\Local\Google\Chrome\User Data\Default\Cookies”.
- En Unix/Linux: “~/.config/google-chrome/Default/Cookies”.
- En macOS: “~/Library/Application Support/Google/Chrome/Default/Cookies”.

Este archivo está en formato SQLite, similar a Firefox, y contiene toda la información sobre las cookies que se han creado mientras navegas por la web. No es legible directamente como texto plano, pero puede ser consultado con herramientas de bases de datos SQLite.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



- En Windows:  
“\Usuarios\<Tu\_usuario>\AppData\Roaming\Opera Software\Opera Stable\Cookies”.
- En Unix/Linux:  
“~/.config/opera/Default/Cookies”
- En macOS: “~/Library/Application Support/com.operasoftware.Opera/Cookies”.



- Windows: “C:\Usuarios\<Tu\_usuario>\AppData\Local\Microsoft\Edge\User Data\Default\Cookies”.
- Unix/Linux: “~/.config/microsoft-edge/Default/Cookies”.
- macOS: “~/Library/Application Support/Microsoft Edge/Default/Cookies”.

Archivo SQLite no legible por editor de texto pero inspeccionable mediante herramientas SQLite.



- macOS :  
“~/Library/Cookies/Cookies.binarycookies”.

Almacenamiento:  
“Cookies.binarycookies” en formato binario.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Opciones de utilización:

Mantener  
opciones de  
visualización.

Almacenamiento  
de variables.

Seguimiento de  
la actividad del  
usuario.

Autenticación de  
usuarios.



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Grabar una cookie:

`document.cookie = "nombreCookie = datosCookie`

`[: expires=horaFormatoGMT]` Ej. `expires=01 Jan 70 00:00:00 GMT;`

`[: path=ruta]` Ruta actual de nuestra página web.

`[: domain=nombreDominio]` Por defecto, el que creó la cookie.

`[: secure]` Solo envía la cookie si la conexión es segura (HTTPS).

`[:HttpOnly]` Impide el acceso a la cookie desde JavaScript, lo que protege contra ataques XSS.

`[:SameSite]"` Limita el uso de la cookie en solicitudes cross-site, mitigando ataques CSRF.

Valores posibles:

**Strict:** Solo envía la cookie en solicitudes del mismo sitio.

**Lax:** Envía la cookie en solicitudes de tipo GET desde otros sitios.

**None:** Permite el envío de cookies en cualquier solicitud (debe combinarse con Secure).

### Borrar una cookie:

Se le asigna una fecha de caducidad anterior a la actual.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Ejemplos:

```
document.cookie = "usuario=Laura; expires=2 Dec 2014, 23:59:59 GMT";
```

```
document.cookie = "contador=0";
```

```
document.cookie = "usuario=Liher; expires=Fri, 31 Dec 2024 23:59:59 GMT; path=/;
domain=ejemplo.com; secure; HttpOnly; SameSite=Lax";
```

*Si la cookie no existe, la crea; si existe, la sobrescribe.*

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Seguridad en las cookies:

- **HttpOnly:** Previene el acceso a las cookies desde JavaScript, protegiendo contra ataques XSS.
- **Secure:** Las cookies solo se envían a través de HTTPS, protegiendo la información en redes inseguras.
- **SameSite:** Limita el envío de cookies a solicitudes del mismo sitio, mitigando CSRF.

## 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

### 5.6. LAS COOKIES



**Función para inicializar una cookie:**

```
function setCookie(cname, cvalue, exdays) {
 var d = new Date(); //Se extrae la fecha actual
 d.setTime(d.getTime() + (exdays*24*60*60*1000)); //Se le suman los días
 indicados por parámetro
 var expires = "expires="+d.toUTCString(); //Se pasa a formato GMT
 document.cookie = cname + "=" + cvalue + "; " + expires; //Se crea la cookie
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Leer una cookie:

**var x = document.cookie; //Devuelve toda la cadena con todas las variables**

**No muestra HttpOnly cookies:** Las cookies con el atributo **HttpOnly** no son accesibles desde JavaScript, por lo que no aparecerán cuando ejecutes `document.cookie`.

### Leer un valor concreto de la cookie:

```
function getCookie(cname) {
 var name = cname + "="; //Crea una variable con el texto del campo y un igual
 var ca = document.cookie.split(';'); //divide la cookie en trozos separados por ;
 for(let i=0; i<ca.length; i++) { //Recorre cada trozo de la cookie
 let c = ca[i].trim(); //Extraemos la cookie y eliminamos el espacio inicial
 if (c.indexOf(name) == 0) //Si encuentra el campo buscado
 return c.substring(name.length,c.length); //Lo devuelve
 }
 return ""; //Si no devuelve vacío
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



**Comprobar si una cookie está inicializada:**

```
function checkCookie() {
 var username=getCookie("username"); //Se extrae el valor (ver función anterior)
 if (username!="") { //Si no está vacío
 alert("Bienvenido de nuevo, " + username); //Se le saluda
 }else{ //Si está vacío
 username = prompt("Introduzca su nombre:"); //Se le solicita con prompt
 if (username != "" && username != null) {
 setCookie("username", username, 365); //Se inicializa la variable
 }
 }
}
```

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.7. WEB STORAGE



### Web Storage

- **API de almacenamiento Web.**
- **Proporciona los mecanismos mediante los cuales el navegador puede almacenar información de tipo clave-valor.**
- **Mucho más intuitiva que utilizando cookies.**
- **El límite de almacenamiento es muy superior al de las cookies (5MB frente a 4KB).**
- **Los datos en Web Storage no se envían automáticamente al servidor en cada solicitud HTTP.**

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES

### Vulnerabilidades comunes de Web Storage:



- **XSS (Cross-Site Scripting):** Los datos en Web Storage son accesibles desde cualquier script en la página. Si un atacante puede inyectar JavaScript malicioso, puede acceder a estos datos.



# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.6. LAS COOKIES



### Seguridad en Web Storage:

- **No almacenar datos sensibles:** Evita almacenar contraseñas, tokens de sesión o información confidencial directamente en localStorage o sessionStorage.
- **Cifrado de datos:** Si necesitas almacenar datos sensibles, asegúrate de cifrarlos antes.  
CDN: `<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-js.min.js"></script>`  
`let cifrado = CryptoJS.AES.encrypt(textoPlano, clave).toString();`  
`let descifrado = CryptoJS.AES.decrypt(cifrado, clave).toString(CryptoJS.enc.Utf8);`
- **Elimina** los datos almacenados cuando ya no sean necesarios.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.7. WEB STORAGE



### Mecanismos de almacenamiento Web

- **sessionStorage**: que permite almacenar información mientras el navegador está abierto; es decir, mientras dura la sesión de la página. Todas las páginas que tienen el mismo origen (dominio y protocolo) pueden acceder a estos datos indistintamente.
- **localStorage**: similar a sessionStorage pero los datos se mantienen aún a pesar de que cerremos el navegador.

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.7. WEB STORAGE



### Métodos

Utilizaremos diferentes métodos que nos permitirán trabajar con este tipo de almacenamiento:

- **setItem**: permite crear un elemento de almacenamiento web.  
`sessionStorage|localStorage.setItem("nombreElemento", "valorElemento");`
- **getItem**: permite consultar un elemento.  
`sessionStorage|localStorage.getItem("nombreElemento");`
- **removeItem** y **clear**: permite eliminar información de un elemento o el Storage completo.  
`sessionStorage|localStorage.removeItem("nombreElemento");`  
`sessionStorage|localStorage.clear();`

# 5. INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

## 5.8. COOKIES VS WEB STORAGE



### Usa Cookies cuando:

- Necesitas enviar datos automáticamente al servidor en cada solicitud HTTP.
- Estás gestionando autenticación o sesiones de usuario.
- Necesitas compartir datos entre subdominios o mantenerlos entre múltiples sesiones.
- Requieres seguridad avanzada con atributos como **HttpOnly**, **Secure**, y **SameSite**.

### Usa Web Storage cuando:

- Estás almacenando datos del lado del cliente que no necesitan ser enviados al servidor en cada solicitud.
- Necesitas mayor capacidad de almacenamiento que las cookies (hasta 5MB o más).
- Estás almacenando datos temporales que solo se necesitan en la sesión actual (con **sessionStorage**).
- Quieres persistir datos del usuario, como preferencias o configuraciones, de forma local (con **localStorage**).
- Estás construyendo aplicaciones que necesitan funcionar sin conexión.